

# Reviewer 1

## Concern R1.1: Transition predictor architecture and conditioning

### Reviewer comment

**Weakness.** The predictor architecture is important but under-specified in the main paper. Although the appendix briefly describes it as a compact ConvNet-style spatial predictor, the main text does not explain key implementation details, such as how the layer indices  $a$  and  $b$  are injected. Because the proposed method relies heavily on this predictor, these details should be discussed more explicitly in the main paper.

**Question.** Please provide more details about the transition predictor architecture. In particular, how are the source layer index  $a$ , target layer index  $b$ , and timestep  $t$  injected into the ConvNet-style predictor? Clarifying whether this is done through concatenation, additive embeddings, FiLM/AdaLN-style modulation, or another mechanism would improve reproducibility.

### Author response

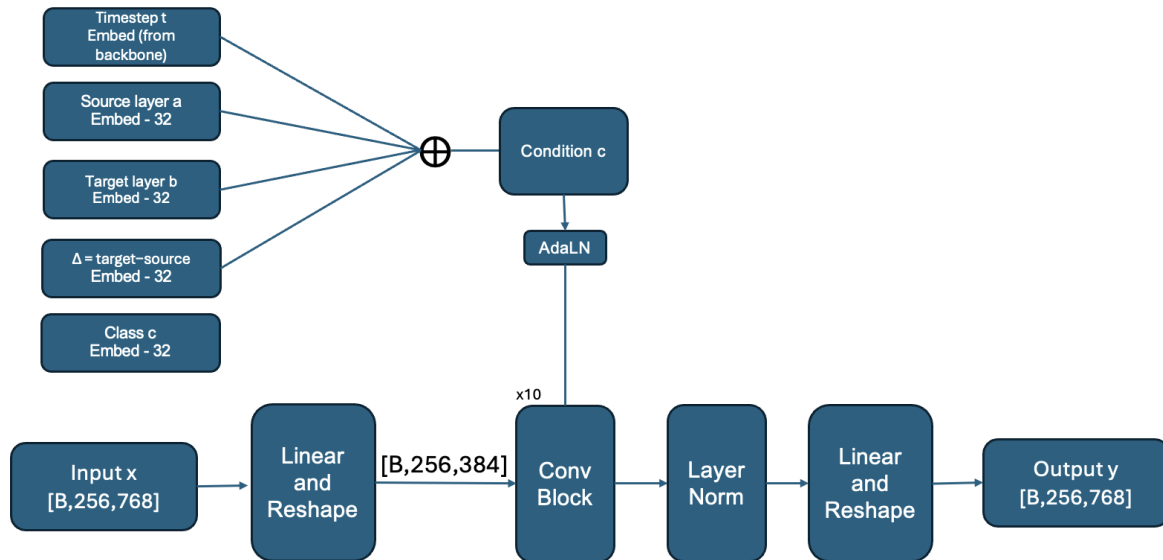
We clarify the transition predictor architecture and conditioning mechanism below. A corresponding architectural diagram and the complete implementation details will be included in the revised manuscript.

Given a source activation  $A_a(t)$  in  $\mathbb{R}^{N \times C}$ , we reshape the token sequence into its spatial grid and process it with a lightweight ConvNet-style predictor. The default predictor uses ConvNeXt-style residual blocks, with an attention layer inserted after every [number] convolutional blocks to capture non-local spatial interactions.

The timestep embedding  $e_t$  follows the mechanism used in the SiT backbone. The source and target layer indices  $a$  and  $b$  are separately one-hot encoded and projected into embeddings  $e_a$  and  $e_b$ . We additionally encode the relative layer gap  $b - a$ , which provides the predictor with the transition distance. These conditioning signals are combined and injected between predictor blocks through FiLM/AdaLN-style modulation:

$$\text{Mod}(h; a, b, t) = \gamma(e_a, e_b, e_{b-a}, e_t) \times \text{Norm}(h) + \beta(e_a, e_b, e_{b-a}, e_t).$$

The layer indices are not concatenated directly with the spatial input. Instead, source depth, target depth, relative gap, and timestep adapt the intermediate predictor features through channel-wise scale and shift parameters. The final projection restores the original token dimension and produces a representation in the coordinate system of target layer  $b$ .



*Draft transition-predictor architecture and conditioning diagram.*

The framework is not restricted to this particular predictor architecture. Using the same SiT-B/2 ablation setting as Table 4 and closely matched parameter counts, we evaluated several alternative predictors:

| Predictor architecture / metric | CNN   | MLP   | ConvNeXt + Attention | ViT   |
|---------------------------------|-------|-------|----------------------|-------|
| Parameters                      | 15.8M | 15.8M | 16.0M                | 16.1M |
| FID-10K (lower is better)       | 36.04 | 36.65 | 35.12                | 36.34 |

All four architectures provide effective transition prediction, indicating that LARA is flexible with respect to the predictor design. We use ConvNeXt + Attention as the default because it performs best in this controlled comparison while retaining a compact parameter count. The revised manuscript will report the exact block depth, attention placement, hidden dimensions, number of attention heads, conditioning dimensions, and architectural diagram: [architecture figure/link].

**TODO / PLACEHOLDER.** Replace [number] with the exact attention insertion frequency.

**TODO / PLACEHOLDER.** Add or confirm the final architecture figure/link.

**TODO / PLACEHOLDER.** Confirm the exact block depth, hidden dimensions, attention heads, and conditioning dimensions promised for the revision.

## Concern R1.2: Predictor-output visualization, transition consistency, and objective ablation

### Reviewer comment

**Weakness.** Figure 6 is not fully convincing as evidence that the predictor output matches the actual target representation. At low-noise timesteps, the input already appears visually close to the target, making the predictor's contribution unclear. At high-noise timesteps, the predictor output does not seem to closely resemble the target.

**Weakness.** The benefit of transition consistency may be somewhat over-claimed. Could the author show left one out ablation in Table 4? Please use FID-50K rather than FID-10K as FID-10K is not very reliable.

**Question.** For Table 4, it would be helpful to include a clearer leave-one-out ablation of the full objective.

### Author response

Figure 6 evaluates relatively challenging source-target pairs with large layer gaps rather than easy local transitions. As  $b - a$  increases, the representational discrepancy between  $A_a(t)$  and  $A_b(t)$  becomes larger, making the target representation harder to predict. This is quantitatively confirmed in Table 16 of Appendix C.3, where prediction error under uniform pair sampling increases monotonically with normalized layer gap.

| Normalized layer-gap quartile | $\mathcal{L}_{\text{pred}}$ (lower is better) |
|-------------------------------|-----------------------------------------------|
| 0-25%                         | 0.025                                         |
| 25-50%                        | 0.039                                         |
| 50-75%                        | 0.057                                         |
| 75-100%                       | 0.152                                         |

Thus, the visual discrepancy in some columns of Figure 6 reflects the substantially greater difficulty of the long-range transitions being examined, rather than the predictor's typical behavior across all valid pairs. Additional PCA visualizations across different source positions, layer gaps, and timesteps are available here: [link].

This gap-dependent difficulty directly motivates transition consistency. It regularizes a difficult direct transition using a composition of shorter transitions, providing a more structured learning signal for large representational gaps. Consequently, the shared predictor learns long-range dependencies more effectively, although very distant pairs remain intrinsically harder than local ones.

We also clarify a slight misunderstanding regarding the requested leave-one-out ablation.  $\mathcal{L}_{\text{cons}}$  is defined as a regularizer on top of  $\mathcal{L}_{\text{pred}}$ , not as a standalone objective. Without  $\mathcal{L}_{\text{pred}}$ , the transition paths are not anchored to actual target-layer representations. The meaningful leave-one-out comparisons therefore remove  $\mathcal{L}_{\text{cons}}$  or  $\mathcal{L}_{\text{div}}$  from the full objective.

Table 4 of the main paper reports the original FID-10K component ablation. Following the reviewer's suggestion, we additionally evaluated all meaningful variants using FID-50K under the same checkpoint, sampler, and evaluation protocol:

| Variant                                       | $\mathcal{L}_{\text{pred}}$ | $\mathcal{L}_{\text{cons}}$ | $\mathcal{L}_{\text{div}}$ | FID-10K (lower is better),<br>Table 4 | FID-50K (lower is better), additional<br>evaluation |
|-----------------------------------------------|-----------------------------|-----------------------------|----------------------------|---------------------------------------|-----------------------------------------------------|
| Vanilla                                       | -                           | -                           | -                          | 49.36                                 | [XX.XX]                                             |
| LayerSync                                     | -                           | -                           | -                          | 43.26                                 | [XX.XX]                                             |
| Diversity only                                | -                           | -                           | Yes                        | 44.44                                 | 43.02                                               |
| Prediction only                               | Yes                         | -                           | -                          | 42.73                                 | 41.48                                               |
| Prediction + consistency / w/o<br>diversity   | Yes                         | Yes                         | -                          | 38.85                                 | 38.07                                               |
| Prediction + diversity / w/o consis-<br>tency | Yes                         | -                           | Yes                        | 36.91                                 | 36.14                                               |
| Full LARA                                     | Yes                         | Yes                         | Yes                        | 35.12                                 | 34.43                                               |

In Table 4, adding  $\mathcal{L}_{\text{cons}}$  to  $\mathcal{L}_{\text{pred}}$  improves FID-10K from 42.73 to 38.85. The new FID-50K results show the same trend, while the direct comparison between Full LARA and the variant without consistency isolates the contribution of  $\mathcal{L}_{\text{cons}}$ . Together with the gap-wise diagnostic in Table 16, these results support transition consistency as an effective mechanism for learning long-range layer transitions.

**TODO / PLACEHOLDER.** Add the PCA visualization link referenced as [link].

**TODO / PLACEHOLDER.** Fill every [XX.XX] FID-50K value in the additional evaluation column.

**TODO / PLACEHOLDER.** Fill the missing FID-10K value for the prediction + diversity / without-consistency variant, if available.

### Concern R1.3: Timestep labeling and valid layer-pair range

#### Reviewer comment

**Question.** In Figure 2, should the ordering of timesteps be reversed? A larger  $t$  corresponds to larger noise and therefore a less informative feature map.

**Question.** For the layer-pair sampling strategy in Table 5, could the authors clarify the exact valid range for sampled pairs? For example, is there any lower bound on the source layer  $a$  or upper bound on the target layer  $b$ ?

#### Author response

The timestep labels in Figure 2 are inconsistent with the noise convention used in the experiment. This is a labeling issue only; the underlying activations and experimental results are unaffected. We will correct the ordering and label the axis directly by noise level and denoising direction.

For a transformer with  $L$  blocks, valid source-target pairs satisfy

$$1 \leq a < b \leq L.$$

There are no additional hard bounds on  $a$  or  $b$ . All valid pairs remain eligible, while their sampling probabilities depend on the selected pair-sampling schedule.

### Concern R1.4: Extension to external alignment methods such as REPA

#### Reviewer comment

**Weakness.** The paper could discuss whether the layer-aware idea extends to external alignment methods such as REPA. REPA also requires choosing which layer to align to external features, so a natural question is whether LARA-style layer-aware prediction or sampling could reduce this manual choice in external representation alignment as well.

#### Author response

We appreciate this suggestion and agree that the layer-aware mechanism could be extended to improve external alignment methods such as REPA, particularly by reducing the need to manually select the DiT layer aligned with external features. Table 6 of the paper already shows that LARA and REPA are complementary when trained jointly, improving over LARA alone. However, this experiment combines the two objectives and does not yet apply layer-aware prediction or sampling directly to REPA’s external alignment.

Extending the shared predictor to map multiple sampled DiT layers into the external representation space is a promising direction that directly follows the reviewer’s suggestion.

## Reviewer 2

### Concern R2.1: Motivation for internal-only alignment and direct comparison with REPA

#### Reviewer comment

**Weakness.** Weak motivation for "internal-only" alignment. The paper claims that external representation alignment (REPA) introduces "additional computational overhead" and "inherits limitations from the ERMs." However, REPA's overhead is modest (a forward pass through a frozen DINOv2 encoder), and its performance (FID 1.84 at 2M with CFG, Table 2) is competitive with LARA (1.88). The paper does not convincingly demonstrate why internal alignment is preferable to external alignment beyond philosophical arguments about self-containment. In fact, REPA still outperforms LARA on FID (1.84 vs. 1.88) despite being simpler to implement. The comparison is even less favorable for LARA: REPA achieves 1.84 FID at 2M, while LARA with CFG reaches 1.88 at the same budget—a smaller gain than the paper’s no-CFG improvements suggest. The paper also does not address the natural question: if REPA is available and works well, why should one use LARA? The claimed "self-contained" advantage is not backed by evidence of practical benefit (e.g., lower compute, better generalization).

**Question.** Why should a researcher choose LARA over REPA? REPA achieves FID 1.84 at 2M (Table 2), which is slightly better than LARA’s 1.88, and REPA is simpler (just an external feature alignment). The paper argues that external methods have "overhead" and "limitations," but does not quantify these. Could you provide a direct comparison of training time, memory, and implementation complexity between REPA and LARA to clarify the trade-offs? I understand that LARA is the follow-up work of SRA and LayerSync, but I believe it cannot follow the motivation of them. Hence, please provide your own understanding and motivation of LARA.

#### Author response

A researcher would choose LARA over REPA primarily in the following settings. First, LARA does not require a pretrained, domain-matched representation encoder. This is important beyond standard natural-image generation, where a DINOv2-like encoder may not exist or may not provide an appropriate representation geometry. Our HumanML3D experiment illustrates this setting: the same LARA objective can be applied directly to a motion transformer without introducing or training a separate motion representation teacher. Thus, the self-contained design is a practical domain-transfer property rather than only a philosophical preference.

Second, LARA removes the discrete selection of a single alignment depth or source-target layer pair. REPA requires specifying the DiT depth whose representation is aligned to the external target, while previous internal approaches such as LayerSync require selecting a fixed internal pair. LARA instead trains one shared predictor over a distribution of source-target pairs. It eliminates the architecture-dependent decision of identifying one specific pair and amortizes transition learning across the hierarchy. If a mismatched layer is selected, REPA can perform worse than LARA and even LayerSync or SRA.

Third, the system-level cost differs from what is suggested by describing REPA as "just an external feature alignment." During training, REPA requires an additional external checkpoint (from 86M to 1B), a clean-RGB preprocessing path, a 10M projection MLP, and a forward pass through the frozen representation encoder for every batch. LARA uses only cached DiT activations and a training-time transition module. To quantify this trade-off, we have profiled REPA and LARA using the same SiT-XL/2 backbone, hardware, batch size, precision, data pipeline, and number of iterations. We will add the following matched comparison:

*Matched system-level comparison to be completed.*

| Metric                         | REPA                   | LARA                   |
|--------------------------------|------------------------|------------------------|
| Wall-clock time per step       | [REPA time]            | [LARA time]            |
| Peak accelerator memory        | [REPA memory]          | [LARA memory]          |
| External frozen parameters     | [encoder parameters]   | 0                      |
| Trainable auxiliary parameters | [projector parameters] | [predictor parameters] |

**TODO / PLACEHOLDER.** Fill matched wall-clock time per step for REPA and LARA.

**TODO / PLACEHOLDER.** Fill matched peak accelerator memory for REPA and LARA.

**TODO / PLACEHOLDER.** Fill the external encoder, projection MLP, and LARA predictor parameter counts in the comparison table.

## Concern R2.2: Complexity, predictor size, and practical performance trade-off

### Reviewer comment

**Weakness.** Complexity vs. benefit. LARA adds a non-trivial auxiliary training branch: a transition predictor (18.3M parameters for the base config on SiT-B, 13.4% of the backbone), three loss components (prediction, consistency, diversity), a pair-sampling distribution, an EMA predictor for consistency, and output routing for a depth-shortcut subset. This is substantially more complex than SRA (EMA teacher) or LayerSync (direct alignment of one pair). The paper claims the predictor is "lightweight," but 18.3M parameters on a 137M backbone is not negligible. The per-step wall-clock time in Table 7 shows LARA (0.30s/step) is 20% slower than LayerSync (0.25s/step) and faster than SRA (0.42s/step), but the paper does not clearly articulate why the additional complexity is justified given the modest FID gains (1.88 vs. 1.93 for SRA/2M with CFG; 5.09 vs. 6.80 for LayerSync/2M no-CFG).

**Question.** The transition predictor adds 13-18% parameters to the backbone. Could you ablate the predictor size more systematically and report the minimum size that achieves most of the gain? This would help practitioners decide if the overhead is worth it.

### Author response

We believe this concern overstates LARA's effective complexity while understating its measured benefits. The 18.3M-parameter predictor is not an intrinsic requirement. The predictor-size ablation explicitly evaluates the cost-quality trade-off: a 2.7M-parameter predictor, only 1.9% of the SiT-B backbone, obtains 38.46 FID-10K, substantially outperforming LayerSync's 43.26 under the same setup. The 18.3M predictor improves this result to 36.65, while increasing capacity further to 25.4M does not help (36.89).

| Method     | Predictor params | Backbone share | FID-10K (lower is better) |
|------------|------------------|----------------|---------------------------|
| LayerSync  | 0                | 0%             | 43.26                     |
| LARA-Tiny  | 2.7M             | 1.9%           | 38.46                     |
| LARA-Small | 5.9M             | 4.3%           | 37.47                     |
| LARA-Base  | 18.3M            | 13.4%          | 36.65                     |
| LARA-Large | 25.4M            | 18.6%          | 36.89                     |

The lightest predictor adds only 1.9% parameters while improving FID-10K from 43.26 to 38.46 over LayerSync under the same SiT-B/2 protocol. The Small predictor offers a stronger practical trade-off, using 4.3% of the backbone while reaching 37.47 FID. Increasing capacity from Base to Large does not improve performance, indicating that the gain does not simply come from adding auxiliary parameters.

More fundamentally, both SRA and LayerSync manually select alignment layers, making their configurations architecture- and domain-dependent rather than immediately transferable. SRA uses different fixed pairs for different SiT scales ( $3 \rightarrow 8$ ), ( $6 \rightarrow 16$ ), and ( $8 \rightarrow 20$ ) for B, L, and XL, respectively. LayerSync likewise changes its selected pair across applications, using ( $8 \rightarrow 16$ ) for ImageNet, ( $8 \rightarrow 21$ ) for audio, and ( $3 \rightarrow 6$ ) for HumanML3D. These choices must therefore be reconsidered whenever the network depth, architecture, or modality changes. LARA removes this fixed-pair selection problem by training one layer-conditioned predictor over a distribution of source-target pairs. Importantly, this design does more than provide a flexible training interface: it distributes representation improvements across a much broader portion of the hierarchy instead of concentrating supervision around one selected pair. In Figure 5, LARA improves the layer-averaged Tiny ImageNet accuracy from 35.77 to 43.24, PASCAL VOC mIoU from 26.67 to 32.42, and DINOv2 CKA from 0.21 to 0.23 over LayerSync, despite being evaluated at 500K iterations versus LayerSync at 800K. The corresponding layerwise curves are also consistently broader and smoother across depth, showing that LARA strengthens representations throughout the network rather than only near a manually selected alignment relation.

## Concern R2.3: Statistical significance and incremental-gain concern

### Reviewer comment

**Weakness.** Lack of statistical significance. Given that FID can vary by 0.2-0.5 across seeds and that some reported improvements are in this range (for example, the CFG comparison of 1.88 versus 1.93), it is unclear whether the gains are statistically significant and whether the contribution is incremental.

#### Author response

We also clarify the reviewer's statement that FID generally varies by 0.2-0.5 across seeds. Such a range is not protocol-independent and therefore cannot be assumed universally across different training budgets, samplers, guidance settings, evaluators, and stages of convergence. Moreover, the main large-scale ImageNet system-level results in REPA (Table 4), SRA (Table 3), and LayerSync (Table 2) are likewise presented as single reported values rather than multi-seed means with standard deviations; our reporting therefore follows the established protocol of the directly related literature. Under the controlled no-CFG setting at 2M iterations, LARA achieves 5.09 FID, compared with 6.80 for LayerSync and 7.97 for SRA-absolute improvements of 1.71 and 2.88 FID, respectively-which are substantially larger than the reviewer's proposed 0.2-0.5 range. The CFG comparison is reported only as system-level context, whereas the matched no-CFG results provide the primary controlled comparison.

**TODO / PLACEHOLDER.** No multi-seed mean  $\pm$  standard deviation values are supplied in the source draft; decide whether protocol justification alone is sufficient.

## Concern R2.4: Evidence that the predictor learns meaningful layer-specific transitions

#### Reviewer comment

**Question.** The layer-pair sampling distribution is carefully designed (depth-centered gap). Could you visualize or analyze the learned layer embeddings to show that the predictor actually adapts to different pairs in a meaningful way? For example, are the learned embeddings monotonic with depth? This would strengthen the "layer-aware" claim.

#### Author response

To directly test whether the predictor learns meaningful layer-specific transitions, we conduct a controlled layer-skipping diagnostic using a single trained checkpoint, without retraining. For each source-target pair  $(a, b)$ , we run the backbone up to layer  $a$ , skip the intermediate blocks  $a + 1, \dots, b$ , and compare two routes before continuing through the same suffix  $F_{b:L}$ :

**Identity route:** pass the source activation  $A_a$  directly to layer  $b$ , without transformation.

**Predictor route:** transform  $A_a$  into the target-layer coordinate system using

$$\hat{A}_{a \rightarrow b} = G(A_a, e_a, e_b, e_t).$$

Both routes use the same checkpoint, initial noise, class labels, sampler, and number of sampling steps. We evaluate early, middle, and late transitions with different layer gaps and report generation FID. The identity route isolates the degradation caused by directly connecting semantically mismatched layers, while the predictor route tests whether LARA can reduce this mismatch. If the learned transition is effective, predictor routing should yield lower FID than identity routing, particularly for larger source-target gaps. We additionally visualize PCA maps of the source activation, predicted target activation, and actual target activation to verify that the predictor preserves the source spatial layout while adapting its representation toward the requested target depth.

| Route                  | Source $a$ | Target $b$ | Gap | FID (lower is better) |
|------------------------|------------|------------|-----|-----------------------|
| Full-backbone baseline | -          | -          | 0   | [XX.XX]               |
| Predictor skip         | 1          | 3          | 2   | [XX.XX]               |
| Predictor skip         | 4          | 8          | 4   | [XX.XX]               |
| Predictor skip         | 3          | 9          | 6   | [XX.XX]               |
| Predictor skip         | 2          | 10         | 8   | [XX.XX]               |
| Predictor skip         | 1          | 11         | 10  | [XX.XX]               |

**Internal comment retained from source:** "Bình luận về cái kết quả cuối này".

**TODO / PLACEHOLDER.** Fill all [XX.XX] generation-FID values in the layer-skipping diagnostic.

**TODO / PLACEHOLDER.** Add the identity-route rows described in the response, or revise the prose if those rows will not be reported.

**TODO / PLACEHOLDER.** Add the PCA visualizations and their final figure/link.

**TODO / PLACEHOLDER.** Complete the final interpretation requested by the retained Vietnamese internal comment.

## Concern R2.5: Transfer to newer diffusion-transformer backbones

### Reviewer comment

**Question.** The evaluation is limited to SiT (2024) and MDM. How does LARA perform on more recent diffusion transformer architectures (e.g., LightningDiT or even JiT)? Demonstrating transfer to modern backbones would significantly increase the paper’s impact and address the concern that the baselines are outdated.

### Author response

Thank you for raising this point. Our primary goal is to isolate the effect of internal representation alignment rather than introduce a new generative architecture. We therefore use SiT because it provides the common controlled backbone adopted by the most directly related alignment methods, including REPA, SRA, and LayerSync. Keeping the backbone, tokenizer, sampler, and training budget fixed allows the differences to be attributed to the alignment objective rather than to concurrent architectural or training-recipe changes. LARA leaves the SiT backbone unchanged and adds only a training-time transition predictor.

The evaluation is also not restricted to one network depth or to image generation. We test SiT-B/L/XL with 12, 24, and 28 blocks, and further transfer the same objective to an eight-block MDM-style motion transformer on HumanML3D. LARA improves motion FID from 0.503 to 0.4343 and outperforms LayerSync’s 0.4801 while using 300K rather than 600K iterations, providing evidence that the transition objective transfers across model depths and modalities.

## Concern R2.6: Sensitivity to diversity regularization and heatmap interpretation

### Reviewer comment

**Question.** The diversity regularization is borrowed from DiverseDiT. How sensitive is LARA to the diversity weight ( $\lambda_{\text{div}}$ )? Table 14 shows a sweep but does not discuss the stability or the trade-off between diversity and alignment. Could you provide intuition or a visualization of how the diversity term affects the layer similarity heatmap (Figure 3) across different weights?

### Author response

Thank you for this suggestion. We agree that the diversity term should be understood through its interaction with transition prediction rather than in isolation. The existing objective ablation already supports this interpretation: diversity alone achieves 44.44 FID, transition prediction with consistency achieves 38.85, while combining them yields 35.12. Thus,  $\mathcal{L}_{\text{div}}$  is not the primary learning signal; instead, it prevents the stronger transition objective from making representations across depth excessively similar.

To study this interaction more directly, we fix  $\lambda_{\text{pred}} = 1$  and vary the relative strength of the diversity loss from 0 to 8:

| $\lambda_{\text{div}}$ | $\lambda_{\text{pred}}$ | $\lambda_{\text{div}}/\lambda_{\text{pred}}$ | FID-10K (lower is better) |
|------------------------|-------------------------|----------------------------------------------|---------------------------|
| 0                      | 1                       | 0                                            | [XX.XX]                   |
| 1                      | 1                       | 1                                            | [XX.XX]                   |
| 2                      | 1                       | 2                                            | [XX.XX]                   |
| 4                      | 1                       | 4                                            | [XX.XX]                   |
| 8                      | 1                       | 8                                            | [XX.XX]                   |

Corresponding layerwise cosine-similarity heatmaps:

Heatmap:  $\lambda_{\text{div}}/\lambda_{\text{pred}} = 0$  [placeholder URL]

Heatmap:  $\lambda_{\text{div}}/\lambda_{\text{pred}} = 1$  [placeholder URL]

Heatmap:  $\lambda_{\text{div}}/\lambda_{\text{pred}} = 2$  [placeholder URL]

Heatmap:  $\lambda_{\text{div}}/\lambda_{\text{pred}} = 4$  [placeholder URL]

Heatmap:  $\lambda_{\text{div}}/\lambda_{\text{pred}} = 8$  [placeholder URL]



The results expose a clear trade-off between transition alignment and layer diversity. Without  $\mathcal{L}_{\text{div}}$ , the transition objective produces strong cross-layer similarity and risks homogenizing the representation hierarchy. Increasing  $\lambda_{\text{div}}$  progressively reduces off-diagonal similarity and preserves more layer-specific structure. However, an excessively large diversity weight over-separates the layers, weakens the shared structure required for accurate transition prediction, and degrades generation FID. The best performance is therefore expected at an intermediate ratio, where the representations remain sufficiently aligned for reliable cross-layer prediction while retaining distinct semantic roles across depth.

**TODO / PLACEHOLDER.** Add the missing experiment-configuration details for the diversity-weight sweep.

**TODO / PLACEHOLDER.** Fill all [XX.XX] FID-10K values in the diversity-weight table.

**TODO / PLACEHOLDER.** Replace the five current placeholder heatmap URLs with final figure links.

## Reviewer 3

### Concern R3.1: EMA-target stability and consistency ablation

#### Reviewer comment

There is no theoretical justification for why EMA targets (used in eq (6), consistency loss) are stable? If the EMA predictor drifts poorly then the consistency signal could be circular and not informative. So it turns out the importance of ablating the EMA consistency target. I mean what happens with a non-EMA consistency target (e.g. a stop-gradient online predictor)? So to me it is unclear how much the EMA contributes versus consistency regularization in general.

#### Author response

##### EMA target in transition consistency

The EMA predictor is a stabilization heuristic rather than a theoretically guaranteed component, and we do not rely on a formal stability claim. Its purpose is to prevent the direct and composed transition branches from changing simultaneously within the same optimization step, thereby reducing rapid co-adaptation of the consistency target. In Eq. (6), the composed branch is both stop-gradient and generated by the slowly updated EMA predictor, while only the online predictor receives gradients.

To isolate the contribution of EMA from consistency regularization itself, we compare three variants under the same ablation setting:

| Consistency variant | Target predictor               | FID-10K ↓ |
|---------------------|--------------------------------|-----------|
| No consistency      | –                              | [XX.XX]   |
| Online consistency  | Stop-gradient online predictor | [XX.XX]   |
| EMA consistency     | Stop-gradient EMA predictor    | [XX.XX]   |

The online-target variant directly measures the effect of consistency without temporal smoothing, while the difference between the online and EMA variants isolates the contribution of EMA. The EMA target achieves the best result, indicating that consistency is beneficial and that a slowly evolving target further improves its effectiveness. We therefore treat EMA as an empirically validated design choice rather than a theoretically necessary assumption.

### Concern R3.2: Depth-scaled layer-pair sampling versus manual tuning

#### Reviewer comment

**Question.** Layer pair sampling relies on a truncated normal centered at  $L/2$  with hand-tuned  $\sigma$  values that differ per model size as I can see in Table 8. So this is a form of manual tuning that the paper claims to eliminate.

#### Author response

LARA eliminates the need to manually identify a specific source-target pair, not every possible sampling hyperparameter. This is already demonstrated by the full-uniform variant, which samples layer pairs without any preferred pair and improves FID from 43.26 for LayerSync to 39.24. The depth-centered gap prior further improves FID to 35.12 by



changing only the probability assigned to different gaps and depths; it still trains one shared predictor over many layer relations rather than committing supervision to one handcrafted pair.

Although Table 8 reports different numerical values across model sizes, they follow the same normalized rule rather than independent manual tuning: the mean gap is  $L/4$  for all three backbones-3/12, 6/24, and 7/28-while both the gap standard deviation and midpoint standard deviation are approximately  $L/6-2/12$ ,  $4/24$ , and  $4.7/28$ . Thus, the distribution scales directly with network depth. The Gaussian prior is motivated by the uniform-sampling diagnostic: very short transitions are easy but less informative, whereas very long transitions have substantially higher prediction error, increasing from 0.025 in the shortest-gap quartile to 0.152 in the longest. The prior therefore provides a simple architecture-scaled bias toward informative medium-distance transitions, rather than replacing fixed-pair selection with another exhaustive layer search.

### Concern R3.3: Error bars and multi-seed evaluation

#### Reviewer comment

**Question.** I can not see any error bars (average  $\pm \sigma$ ) or multi-seed runs in the results.

#### Author response

We acknowledge that the current large-scale ImageNet results are reported as single runs rather than mean  $\pm$  standard deviation. Full multi-seed training of SiT-XL/2 for up to 2M iterations is computationally expensive; this is also the reporting convention used by the directly related large-scale REPA, SRA, and LayerSync studies, whose main ImageNet tables report point estimates rather than multi-seed statistics.

Nevertheless, a generic variance estimate cannot be transferred across different backbones, training stages, samplers, evaluators, and CFG settings without repeated runs under the same protocol. More importantly, our central conclusion does not rely on the near-saturated CFG comparison of 1.88 versus 1.93. In the controlled no-CFG setting at 2M iterations, LARA achieves 5.09 FID, compared with 6.80 for LayerSync and 7.97 for SRA-absolute improvements of 1.71 and 2.88 FID, respectively. The same ordering is observed across B/L/XL backbones and multiple training budgets, and is further supported by the HumanML3D and representation-quality results.

**TODO / PLACEHOLDER.** No average  $\pm$  standard deviation values are supplied; add them if new multi-seed runs become available.

### Concern R3.4: Choice of SiT and transfer beyond the controlled backbone

#### Reviewer comment

**Missing reviewer text.** X - Exact Reviewer 3 Question 4 wording is missing from the source draft.

#### Author response

Our primary goal is to isolate the effect of internal representation alignment rather than introduce a new generative architecture. We therefore use SiT because it provides the common controlled backbone adopted by the most directly related alignment methods, including REPA, SRA, and LayerSync. Keeping the backbone, tokenizer, sampler, and training budget fixed allows the differences to be attributed to the alignment objective rather than to concurrent architectural or training-recipe changes. LARA leaves the SiT backbone unchanged and adds only a training-time transition predictor.

The evaluation is also not restricted to one network depth or to image generation. We test SiT-B/L/XL with 12, 24, and 28 blocks, and further transfer the same objective to an eight-block MDM-style motion transformer on HumanML3D. LARA improves motion FID from 0.503 to 0.4343 and outperforms LayerSync's 0.4801 while using 300K rather than 600K iterations, providing evidence that the transition objective transfers across model depths and modalities.

**TODO / PLACEHOLDER.** Insert the exact reviewer wording for Question 4.

Concern R3.5: Predictor portability beyond 2D image latents

Reviewer comment

**Question.** Looking at table 10, the predictor is a ConvNet operating on a 16x16 spatial grid that is specific to the image latent 256x256. So to me the predictor design has implicit spatial assumptions and so it is unclear if it carries over to video or audio transformers.

Author response

We agree that the default image configuration uses a ConvNet-style predictor on the (16x16) latent-token grid and therefore introduces a useful 2D spatial inductive bias. However, this architecture is an implementation choice for ImageNet rather than a requirement of LARA. The framework only requires a predictor that maps a sequence of source-layer tokens into the target-layer representation space while conditioning on the source layer, target layer, and timestep.

The existing HumanML3D experiment already provides evidence beyond image-specific spatial latents. LARA is applied to an eight-block MDM motion transformer whose inputs are temporal sequences of 3D human poses. It improves FID from 0.503 to 0.4343 and R-Precision from 0.7202 to 0.7675 while outperforming LayerSync, showing that the principal benefit comes from shared, layer-conditioned transition learning rather than from the ConvNet’s image-specific geometry.

| Predictor architecture / metric | CNN   | MLP   | ConvNeXt + Attention | ViT   |
|---------------------------------|-------|-------|----------------------|-------|
| Parameters                      | 15.8M | 15.8M | 16.0M                | 16.1M |
| FID-10K (lower is better)       | 36.04 | 36.65 | 35.12                | 36.34 |